

AWS SENIOR ENGINEER INTERVIEW PREPARATION

Part 3 — Questions 101–150

SQS · SNS · EventBridge · Step Functions · ECS/Fargate
DynamoDB · ElastiCache · CloudFormation · CDK · Cost Optimization
CloudWatch · X-Ray · Secrets Manager · Parameter Store · CI/CD on AWS

50 Scenario-Based Questions · Expert-Level Answers · Follow-Up Probes

Document	Series	Questions	Difficulty Mix	Date
AWS Interview Questions — Part 3	Part 3 of 3 (Q101– Q150)	50 scenario-based questions	Easy: 6 Medium: 26 Hard: 18	March 2026

Topics Covered in Parts 1 & 2 (do not repeat):

Part 1 (Q1–Q50): EC2, VPC, ALB/NLB, ASG, RDS (MySQL/Postgres), S3, Lambda, Route 53, CloudFront, Security Groups / NACLs, EKS basics.

Part 2 (Q51–Q100): EKS advanced, IAM (policies, SCPs, IRSA, JIT), Advanced networking (TGW, PrivateLink, Network Firewall), S3 advanced (data lake, sovereignty), RDS advanced (encryption, Proxy, replicas), ML workloads, Multi-AZ design.

Part 3 covers:

SQS, SNS, EventBridge, Step Functions, ECS/Fargate, DynamoDB, ElastiCache (Redis/Memcached), CloudFormation/CDK, Cost Optimization, CloudWatch/X-Ray observability, Secrets Manager/Parameter Store, and AWS CI/CD (CodePipeline/CodeBuild/CodeDeploy).

Messaging & Event-Driven Architecture

SQS · SNS · EventBridge · Step Functions

Q101

SQS — Queue Design

Hard

Your application consumes from an SQS Standard queue. Messages are being processed, but occasionally the same message is processed twice, causing duplicate database entries. The team says they enabled 'idempotent processing' but duplicates still appear. What's going wrong and how do you fix it?

 **Testing:** SQS visibility timeout mechanics, at-least-once delivery, idempotency implementation

✓ Key Answer Points:

- SQS Standard guarantees at-least-once delivery — duplicate delivery is expected, not a bug
- Visibility timeout: when a consumer reads a message, it becomes invisible for visibilityTimeout seconds. If NOT deleted in time: becomes visible again → second consumer processes it
- Root cause of 'idempotency not working': likely the de-duplication check happens AFTER the DB write, not before
- Fix 1: use idempotency key — store a processed_message_ids table in DB/Redis. Before processing: check if message_id exists. If yes: skip. If no: process + insert + mark processed
- Fix 2: SQS FIFO queue with MessageDeduplicationId — FIFO guarantees exactly-once within 5-minute dedup window
- Fix 3: increase visibilityTimeout to be longer than max processing time — reduces timeout-induced redelivery
- CloudWatch metric: ApproximateNumberOfMessagesNotVisible (in-flight count) vs NumberOfMessagesSent
- DLQ: after maxReceiveCount attempts, send to DLQ — investigate DLQ for poison messages

 **Follow-up:** What's the difference between SQS Standard and FIFO in terms of throughput limits, and when would you NOT use FIFO?

Q102

SQS — Consumer Scaling

Medium

You have a Lambda function triggered by SQS. During peak load, messages build up in the queue and processing latency increases. The Lambda timeout is 15 minutes but most messages take 2 seconds. How do you diagnose and fix this?

 **Testing:** SQS-Lambda integration internals, batch size, concurrency, scaling behaviour

✓ Key Answer Points:

- SQS-Lambda event source mapping: Lambda service polls SQS and invokes Lambda with batches
- Default: 5 pollers × 1 batch = 5 concurrent Lambdas. Scales up by adding 60 pollers/minute until queue drains
- Problem: Lambda account concurrency limit (default 1000) may be reached — throttling causes queue backup
- Fix 1: increase batchSize (up to 10,000 for Standard, 10 for FIFO) — process more messages per invocation
- Fix 2: increase Lambda reserved concurrency for this function to prevent other functions stealing quota
- Fix 3: if processing is CPU-bound, consider ECS/Fargate consumers instead — no concurrency limits
- Fix 4: SQS Lambda scaling: set maximumConcurrency on the event source mapping (Lambdas 1.0.315+)
- Monitor: ApproximateAgeOfOldestMessage (queue depth latency proxy) — set alarm on this, not queue size
- Lambda batch window (maxBatchingWindow): wait up to N seconds to fill a batch — reduces Lambda invocations

 **Follow-up:** If a Lambda within a batch fails, what happens to the entire batch? How does bisect-on-error change this?

Q103

SNS — Fan-Out Pattern

Medium

3

Your e-commerce platform needs to: (1) update inventory, (2) send a confirmation email, (3) trigger fraud detection — all when an order is placed. Each system is independent and developed by different teams. Design this using AWS messaging services.

 **Testing:** SNS fan-out pattern, topic subscriptions, SQS as buffer, decoupling design

 Key Answer Points:

- Pattern: SNS topic → multiple SQS queue subscriptions (fan-out)
- OrderPlaced event → SNS topic 'order-events'
- SNS delivers to 3 subscribers: SQS-inventory, SQS-email, SQS-fraud
- Each team owns their own SQS queue + consumer service — fully decoupled
- Why SQS between SNS and consumers: buffers spikes, allows retries, each team controls their own DLQ
- SNS message filtering: each subscription can filter by MessageAttribute — e.g., fraud only cares about orders > \$1000
- SNS FIFO + SQS FIFO: if strict ordering matters across all consumers
- Alternative: EventBridge if schema evolution and event bus routing flexibility is needed
- Monitor each SQS queue independently — inventory queue backup doesn't affect email queue
- SNS dead-letter queue: for failed deliveries to HTTP/Lambda subscribers (not SQS subscribers)

 **Follow-up:** What happens if the inventory SQS queue's consumer is down for 2 hours? How does the fan-out pattern handle this gracefully?

Q10
4

EventBridge — Event Routing

Medium

You're replacing a monolithic event system with EventBridge. The old system has 40 different event types, 15 consumers, and some consumers need events from multiple sources (S3, RDS, custom application). How do you design the EventBridge architecture?

 **Testing:** EventBridge concepts: event buses, rules, targets, schema registry, cross-account

 Key Answer Points:

- Event buses: use custom bus per domain (orders-bus, inventory-bus, user-bus) — not everything on default bus
- Rules: each consumer defines its own rule with event pattern filtering — consumer owns their subscription
- Event pattern: filter by source, detail-type, or any field in detail — consumers only receive what they need
- Schema registry: auto-discover schemas from events; generate code bindings for each language
- Cross-account: EventBridge can route to targets in other accounts — use resource-based policy on target bus
- S3 events → EventBridge: enable S3 EventBridge notifications (not S3 Event Notifications to SQS/Lambda)
- RDS events → EventBridge: via CloudTrail or RDS Event Subscriptions → SNS → EventBridge
- Custom app: PutEvents API with detailType and source identifying the event
- DLQ on rules: if target fails after retries, send to DLQ for investigation
- EventBridge Pipes: for simple point-to-point with filtering/enrichment (SQS → Lambda with filter)

 **Follow-up:** How does EventBridge schema registry help with event versioning and breaking changes between producers and consumers?

Q10
5

Step Functions — Workflow Orchestration

Hard

You have a multi-step order processing workflow: validate payment (3rd party API, <1s), reserve inventory (internal DB, <100ms), create shipment (3rd party API, 2-5s), send notification (SES, <1s). Occasionally the shipment API is down for 5-10 minutes. Design this as a Step Functions workflow with proper error handling.

🎯 Testing: Step Functions state machine design, error handling, retries, wait states, Express vs Standard

✅ Key Answer Points:

- Standard Workflow (not Express): order processing needs audit trail, longer duration, exactly-once execution
- States: ValidatePayment → ReserveInventory → CreateShipment → SendNotification
- ReserveInventory should use DynamoDB conditional writes (optimistic locking) — idempotent
- CreateShipment: configure Retry with intervalSeconds:30, maxAttempts:10, backoffRate:1.5 for 5xx errors
- CreateShipment: configure Catch for non-retryable errors → ReleaseInventory (compensating transaction)
- Saga pattern: each step has a compensating action. ValidatePayment fails → nothing to undo. ReserveInventory fails → nothing to undo. CreateShipment fails → ReleaseInventory then NotifyFailure
- Wait state for async: if CreateShipment is async, use .waitForTaskToken — Lambda calls back with token when done
- Express Workflow: for high-volume, short-duration workflows (not here — need exactly-once and audit)
- Step Functions SDK integration: call DynamoDB/SES directly without Lambda — reduces cost and latency
- Monitoring: Step Functions execution history in console; CloudWatch metrics per state; X-Ray tracing

🔄 **Follow-up:** What's the difference between Express and Standard workflows in terms of execution guarantees, and which would you use for a high-frequency IoT data processing pipeline?

Q106

SQS — Dead Letter Queue Strategy

Medium

Your team keeps finding messages in the DLQ but has no systematic process for investigating them. Messages just sit there. Management asks for a proper DLQ handling strategy. What do you design?

🎯 Testing: DLQ operational patterns, message investigation, replay, monitoring

✅ Key Answer Points:

- DLQ is a signal, not a storage: every DLQ message represents a processing failure that needs investigation
- Alert: CloudWatch alarm on ApproximateNumberOfMessagesVisible in DLQ > 0 → immediate notification
- Investigation tooling: Lambda subscribed to DLQ (visibilityTimeout=0) that logs full message + metadata to CloudWatch Logs/S3 for analysis — does NOT delete the message
- Root cause categories: (1) code bug in consumer, (2) malformed message from producer, (3) dependency outage, (4) message too large/old
- Redrive: after fixing the bug, use SQS Redrive (console or API) to move messages back to source queue
- Redrive policy: set maxReceiveCount carefully — 3 for fast-fail, 5-10 for dependency retries
- SQS Message retention: default 4 days, max 14 days. Set DLQ retention to 14 days to give time to investigate
- Lambda DLQ vs SQS DLQ: Lambda DLQ catches async invocation failures (not event source mapping failures — those retry in SQS)
- Dashboard: DLQ depth per service, time-in-DLQ, redrive rate, recurring failures by message attribute

🔄 **Follow-up:** How would you design an automated DLQ redrive that only replays messages after a code fix is deployed, preventing re-failure?



Container Orchestration

ECS · Fargate · ECR · Service Discovery

Q10
7

ECS — Fargate Task Sizing

Medium

Your Fargate tasks are consistently OOMKilled. CPU usage is low (10-15%) but memory hits 100% and the task is killed. The task definition has 512 MB memory. How do you diagnose and right-size?

Testing: Fargate resource model, container memory limits, CloudWatch Container Insights

Key Answer Points:

- Fargate memory model: task-level memory is hard limit. Container gets killed if it exceeds task memory minus OS overhead
- 512 MB task = ~470 MB available to containers (Fargate reserves ~30-50 MB for OS/agent)
- Diagnosis: CloudWatch Container Insights → MemoryUtilized metric per task/service over time
- Identify peak: look for 99th percentile memory usage, not average — size for peak
- Java gotcha: JVM doesn't respect cgroup memory limits by default. Use -XX:+UseContainerSupport (Java 10+) or -Xmx flag
- Node.js gotcha: V8 default heap size doesn't know about container limits — set --max-old-space-size explicitly
- Fix: increase task memory in steps (512 → 1024 → 2048). vCPU and memory must be valid Fargate combinations
- Fargate valid combinations: 0.25 vCPU: 512MB–2GB | 0.5 vCPU: 1–4GB | 1 vCPU: 2–8GB | 2 vCPU: 4–16GB | 4 vCPU: 8–30GB
- Add memory reservation (soft limit) + memory limit (hard) to container definition separately
- Cost: Fargate pricing is per-vCPU-second and per-GB-second — right-sizing reduces cost directly

Follow-up: How does Fargate Spot change your architecture and what happens when a Spot task is interrupted mid-processing?

Q10
8

ECS — Service Discovery & Load Balancing

Medium

You're migrating a microservices application from EC2 to ECS Fargate. Services need to communicate with each other. Some services are internal-only. How do you handle service discovery and routing between services?

Testing: ECS service discovery options: Cloud Map, ALB, internal DNS, App Mesh

Key Answer Points:

- Option 1: ALB per service (HTTP) — use internal ALB for service-to-service. DNS: myservice.internal. Simple but costly at scale (one ALB per service minimum)
- Option 2: AWS Cloud Map (Service Discovery) — ECS registers task IPs in Cloud Map. Consumer resolves myservice.namespace via Route 53 private DNS. Direct task-to-task; no LB; uses health checks
- Option 3: App Mesh — sidecar Envoy proxy on each task. mTLS between services, circuit breaking, retries, traffic shifting. Most powerful but most complex
- Recommended for most teams: Cloud Map for simple service discovery + ALB only for services needing LB features
- ECS + Cloud Map setup: in service definition, enable serviceRegistries pointing to Cloud Map service
- Internal ALB: scheme=internal; DNS in Route 53 private hosted zone; accessible only within VPC
- Health checks: Cloud Map uses customHealthCheck; ALB uses target group health checks
- VPC DNS: enable enableDnsHostnames and enableDnsSupport on VPC — required for Cloud Map resolution
- Port mapping: Fargate uses awsvpc mode — each task gets its own ENI and IP — Cloud Map registers this IP

Follow-up: If you use Cloud Map and 5 instances of Service A are registered, does Cloud Map do load

balancing? If not, how do clients distribute requests?

Q109

ECS — Rolling Deploy & Rollback

Hard

You're doing a rolling deploy of a critical ECS Fargate service (10 tasks). The new version has a bug — 30% of requests return 500. By the time you notice, 6 tasks are running new version, 4 are old. Production error rate is ~18%. How do you handle this?

 **Testing:** ECS deployment mechanics, circuit breaker, rollback, blue/green with CodeDeploy

 Key Answer Points:

- Immediate: trigger a rollback in ECS console or CLI: `aws ecs update-service --task-definition <previous-revision>`
- ECS rolling deploy: min 100% healthy, max 200% capacity — new tasks start before old stop
- With deployment circuit breaker enabled: ECS automatically rolls back if new tasks fail health checks
- Circuit breaker threshold: if N% of tasks fail during deployment → rollback triggered automatically
- Blue/Green with CodeDeploy: eliminates this scenario — traffic shifted only after health checks pass on green fleet
- B/G mechanics: CodeDeploy creates new task set, runs health checks, shifts traffic (linear/canary/all-at-once), terminates old task set after stability period
- Canary deploy: shift 10% traffic to new version → monitor error rate → promote or rollback
- For immediate recovery: `aws ecs update-service --service myservice --task-definition myservice:42` (previous revision)
- Container Insights + ALB access logs: identify the moment error rate spiked and correlate with deployment event
- Prevent: enable ECS deployment circuit breaker with `rollback: true` in service definition

 **Follow-up:** How does ECS Blue/Green with CodeDeploy handle stateful services (e.g., services with sticky sessions or active WebSocket connections)?

Q110

ECR — Image Lifecycle & Security

Medium

Your ECR repository has grown to 2TB and costs \$200/month. Additionally, a security scan found 3 critical CVEs in an image that's currently deployed in production. How do you address both the cost and security issues?

 **Testing:** ECR lifecycle policies, image scanning, vulnerability management, patch strategy

 Key Answer Points:

- Cost: ECR lifecycle policy — automatically expire old images
- Policy example: keep only last 10 images per tag prefix; expire untagged images after 1 day; expire non-production images after 30 days
- Apply lifecycle policy via console or: `aws ecr put-lifecycle-policy --repository-name myapp --lifecycle-policy-text file://policy.json`
- Security — CVE remediation: identify which base image layer introduced the CVE (ECR Scan findings show layer)
- Fix: update base image in Dockerfile (e.g., `FROM node:18-alpine` → `FROM node:20-alpine`), rebuild, push, redeploy
- ECR Enhanced Scanning (Inspector): continuous scanning — alerts when new CVE affects existing images even without new push
- ECR Basic Scanning: on-push only — won't alert on newly discovered CVEs for old images
- Enforce: ECR image tag mutability = `IMMUTABLE` — prevents overwriting a tag (critical for supply chain security)
- Block deployment of vulnerable images: use OPA/Kyverno admission webhook that checks ECR scan status before allowing pod creation

- SBOMs: enable ECR to generate Software Bill of Materials for compliance

 **Follow-up:** *What is ECR image tag immutability, and how do you implement a 'latest always safe' policy while keeping immutability enabled?*



DynamoDB

NoSQL design, scaling, hot partitions, streams, Global Tables

Q11
1

DynamoDB — Table Design

Hard

You're designing a DynamoDB table for a multi-tenant SaaS application. Each tenant has users, orders, and products. A tenant can have up to 1M orders. The application needs to: list all orders for a tenant, get a specific order, list orders by status, and get all users for a tenant. Design the table.

Testing: Single-table design, partition key, sort key, GSI design, access pattern-first thinking

Key Answer Points:

- Single-table design: store all entity types in one table. PK and SK carry semantic meaning
- PK = TENANT#{tenantId}, SK = ORDER#{orderId} for orders → enables 'list all orders for tenant' efficiently
- PK = TENANT#{tenantId}, SK = USER#{userId} for users → enables 'list all users for tenant'
- PK = TENANT#{tenantId}, SK = PRODUCT#{productId} for products
- Get specific order: PK = TENANT#{tenantId}, SK = ORDER#{orderId} → direct lookup O(1)
- Orders by status: GSI with PK = TENANT#{tenantId}#STATUS#{status}, SK = ORDER#{orderId} → list orders by status per tenant
- Attribute names: avoid using reserved words; prefix entity-type in SK makes range queries deterministic
- Avoid hot partitions: if one tenant has 1M orders, that partition key handles huge traffic. Solution: distribute across multiple partitions with a suffix (shard) if write rate is > 1000 WCU/s for one tenant
- TTL: add TTL attribute to orders for automatic expiration (e.g., archive after 7 years)
- DynamoDB Streams + Lambda: capture changes for downstream systems without polling

Follow-up: How would you handle a report query that needs to aggregate all orders across ALL tenants for the last 30 days? DynamoDB can't do this efficiently — what's your architecture?

Q11
2

DynamoDB — Hot Partition & Throttling

Hard

Your DynamoDB table has 100 provisioned RCU. CloudWatch shows ConsumedReadCapacityUnits averaging 60 RCU but you're getting ProvisionedThroughputExceededException. How is this possible, and how do you fix it?

Testing: DynamoDB partition heat, adaptive capacity, burst capacity, partition-level throttling

Key Answer Points:

- Key insight: provisioned throughput is distributed across partitions. A table with 100 RCU split across 10 partitions = 10 RCU/partition maximum
- Hot partition: if most reads hit 1-2 partitions (e.g., popular products, recent orders), those partitions exhaust their share even when table average is low
- Adaptive Capacity (automatic): DynamoDB can lend throughput from cold partitions to hot ones — but has limits and lag
- Burst capacity: DynamoDB retains up to 5 minutes of unused capacity per partition — only delays the problem
- Fix 1: Switch to On-Demand mode — DynamoDB handles partition distribution automatically, no capacity planning
- Fix 2: Add write sharding for hot partition keys — append random suffix (0-9) to PK, read from all shards and merge
- Fix 3: DAX (DynamoDB Accelerator) — in-memory cache in front of DynamoDB, reduces read traffic to hot partitions
- Fix 4: Application-level caching (ElastiCache) for frequently accessed items
- Detect: CloudWatch → ConsumedReadCapacityUnits per partition is NOT available directly. Use CloudWatch Contributor Insights for DynamoDB to find hot keys
- CloudWatch Contributor Insights: shows top N most accessed partition keys and sort keys — enables hot key

identification

 **Follow-up:** When would you use DAX vs ElastiCache as a DynamoDB caching layer, and what are the consistency trade-offs of each?

Q11
3**DynamoDB — Streams & Global Tables****Medium**

You need to replicate data from your DynamoDB table in us-east-1 to eu-west-1 for disaster recovery, with RPO < 1 second. You also need to trigger downstream processing when items change. Design the solution.

 **Testing:** DynamoDB Global Tables, Streams, Lambda triggers, multi-region architecture

 Key Answer Points:

- Global Tables: enable multi-active replication across regions. Writes in any region replicate to all others
- Global Tables replication: uses DynamoDB Streams internally; typical replication lag < 1 second
- Enable: create table in both regions, then enable Global Tables replication — DynamoDB manages sync
- Conflict resolution: last-writer-wins based on timestamp — design application to avoid concurrent cross-region writes to same item
- DynamoDB Streams for downstream processing: enable on the table, create Lambda trigger
- Stream record types: INSERT, MODIFY, REMOVE — Lambda receives batch of stream records
- Stream view types: KEYS_ONLY (cheapest), NEW_IMAGE, OLD_IMAGE, NEW_AND_OLD_IMAGES (most expensive but most useful for auditing)
- Lambda + Streams: Lambda polls the stream. Configure batch size, parallelism factor (per shard), bisect-on-error
- Use case: Stream → Lambda → Elasticsearch for full-text search on DynamoDB data
- Stream retention: 24 hours. If Lambda is down > 24h, records are lost — monitor iterator age

 **Follow-up:** If you have Global Tables active in both us-east-1 and eu-west-1 and a network partition occurs between regions for 5 minutes, what happens to writes? How do you detect and resolve conflicts after reconnection?

Q11
4**DynamoDB — Cost Optimisation****Medium**

Your DynamoDB costs jumped from \$500 to \$4,000/month. Nothing changed in the application. How do you investigate and reduce costs?

 **Testing:** DynamoDB cost model, Cost Explorer, scan vs query, capacity mode selection

 Key Answer Points:

- Investigation: AWS Cost Explorer → DynamoDB → break down by usage type (ReadCapacityUnit-Hrs, WriteCapacityUnit-Hrs, Storage, BackupStorage, Replication)
- Common causes of cost spike: (1) accidental table scan in application code, (2) new backup/PITR enabled, (3) Streams data transfer, (4) Global Table replication, (5) on-demand mode with traffic spike
- CloudWatch: ConsumedReadCapacityUnits spike → check application for table scans (Scan API is expensive)
- Scan vs Query: Scan reads EVERY item in table (costs entire table's RCU). Query uses index (cheap). Even filtered Scan reads everything before filtering
- Check code: grep for DynamoDB Scan API calls — replace with Query with proper index
- On-Demand vs Provisioned: On-Demand is ~6x more expensive per operation at steady load. Switch to Provisioned with auto-scaling for predictable workloads
- Auto-scaling: set min/max RCU/WCU, target utilization 70% — scales within 1-2 minutes
- PITR cost: 0.20\$/GB/month for backup storage. Disable if not needed; use on-demand backups instead
- TTL: enable TTL on expired items — free deletion, reduces storage cost
- Compress large attributes: DynamoDB charges per-item size. Compress JSON attributes with gzip before

storing

 **Follow-up:** *How does DynamoDB on-demand pricing work during a flash sale where you get 100x normal traffic for 2 hours? When does it become more expensive than provisioned + auto-scaling?*

⚡ ElastiCache

Redis · Memcached · Caching patterns · High Availability

Q11
5

ElastiCache Redis — Cluster Design

Hard

Your Redis cluster is running out of memory. You have a single-node Redis with 16 GB. The dataset is 14 GB and growing 500 MB/week. Additionally, the Redis node went down for 20 minutes last month causing a production outage. Design a production-grade Redis architecture.

🎯 **Testing:** Redis Cluster mode, replication groups, Multi-AZ, eviction policies, ElastiCache sizing

✓ Key Answer Points:

- Problem 1 (memory): single node will fill up in ~4 weeks at current growth
- Problem 2 (availability): single node = single point of failure. When it died, no failover
- Solution: ElastiCache Redis Cluster Mode Enabled
- Cluster mode: horizontally shards data across multiple primary nodes (e.g., 3 shards × 16 GB = 48 GB total)
- Each shard: 1 primary + 1-5 replicas. Multi-AZ: primary and replica in different AZs
- Automatic failover: if primary dies, replica promoted in ~20-30 seconds (not 20 minutes)
- Migration path: ElastiCache Online Replication — online migration from non-cluster to cluster mode
- Eviction policy: set maxmemory-policy. Options: allkeys-lru (evict least recently used — good for cache), volatile-lru (only evict keys with TTL set), noeviction (reject writes when full — dangerous for cache)
- Monitoring: CurrConnections (connection exhaustion), CacheHitRate (should be > 95%), Evictions (if > 0, cache is memory-pressured), ReplicationLag (replica is behind)
- Redis AUTH + TLS in transit + encryption at rest — enable all three
- Reserved nodes: commit to 1 or 3 year reserved pricing — 30-60% savings over on-demand

🔄 **Follow-up:** How does Redis Cluster mode handle multi-key operations (MGET, transactions, Lua scripts) that span multiple shards? What application changes are required?

Q11
6

ElastiCache — Caching Strategies

Medium

You're adding caching to a read-heavy application backed by RDS. The product catalog (10K items, rarely changes) is queried 50K times/second. User sessions (1M users, change frequently) are stored in the DB. Design the caching strategy for both use cases.

🎯 **Testing:** Cache-aside vs write-through vs write-behind, TTL strategy, cache invalidation

✓ Key Answer Points:

- Product catalog — Cache-aside (lazy loading): cache hit → return from Redis. Cache miss → query RDS, populate Redis, return result
- Product catalog TTL: 1 hour (data rarely changes). On update: explicitly invalidate the cache key OR use write-through
- Write-through for catalog: on every DB write, also write to Redis. Cache always warm. Risk: write penalty + cache pollution if many items written but not read
- Cache key design: product:{productId} → single item. product:list:{category}:{page} → paginated lists
- User sessions — Redis as primary session store (not DB): write session to Redis directly, not RDS
- Session TTL: match session expiry (e.g., 24h or 30-day remember-me). Redis TTL handles expiry automatically
- Session data structure: Hash (HSET/HGET) for partial updates without full read-modify-write
- Cache invalidation strategy: (1) TTL-based expiry, (2) event-driven invalidation (SQS/EventBridge on catalog update → Lambda → Redis DEL)
- Cache stampede / thundering herd: when TTL expires, many requests hit RDS simultaneously. Fix: cache lock pattern or probabilistic early expiration
- Connection pooling: application should pool Redis connections. Each Lambda invocation creates a new

connection — use Lambda Redis connection reuse (define client outside handler)

 **Follow-up:** How do you handle cache invalidation across multiple Redis nodes in a cluster when a product is updated? How do you ensure all nodes invalidate consistently?

Q11
7

ElastiCache — Redis vs Memcached

Easy

A developer asks: 'We need a cache. Should we use Redis or Memcached?' How do you answer, and what questions do you ask first?

 **Testing:** Feature comparison, use case selection, architectural trade-offs

✓ Key Answer Points:

- Questions to ask: (1) Do you need persistence? (2) Do you need data structures beyond strings? (3) Do you need pub/sub? (4) Do you need clustering with sharding? (5) Multi-threading requirement?
- Choose Redis when: need persistence (RDB/AOF), need data structures (lists, sets, sorted sets, hashes, HyperLogLog, streams), need pub/sub messaging, need Lua scripting, need Cluster mode for sharding
- Choose Memcached when: pure caching with no persistence needed, simplest possible setup, need multi-threaded performance (Memcached is multi-threaded, Redis single-threaded main loop), horizontal scaling via multiple nodes without replication
- Redis sorted sets: leaderboards (ZADD/ZRANGE), rate limiting (sliding window), time-series data
- Redis Streams: append-only log, consumer groups — replaces Kafka for simple messaging
- In practice: Redis is almost always the right choice — more features, widely understood, ElastiCache manages HA
- Memcached advantage: truly multi-threaded — at very high connection counts (>10K), Memcached can outperform Redis. Redis 7.0+ added I/O threading but main command processing is still single-threaded
- AWS recommendation: if you need backup, replication, Pub/Sub — Redis. If you need simple, fast, horizontally scalable cache only — Memcached

 **Follow-up:** What is Redis Sentinel, and how does ElastiCache Redis with Multi-AZ differ from a self-managed Redis Sentinel setup?



Infrastructure as Code

CloudFormation · CDK · Drift Detection · StackSets

Q11
8

CloudFormation — Stack Update Safety

Hard

You're about to deploy a CloudFormation stack update that modifies an RDS instance (instance class change), an S3 bucket (add lifecycle policy), and a Lambda function (new code). What is your deployment strategy to minimize risk and downtime?

Testing: CloudFormation change sets, update behavior (no interruption/some interruption/replacement), rollback

✓ Key Answer Points:

- First: create a Change Set — never deploy without reviewing the change set
- Change Set review: look for 'Replacement: True' — this means the resource will be DELETED and RECREATED
 - RDS instance class change: 'Requires: Some interruption' — RDS will reboot (1-5 min downtime). Plan for a maintenance window
 - S3 lifecycle policy: 'Requires: No interruption' — safe to apply anytime
 - Lambda function: 'Requires: No interruption' — code update is atomic, no downtime
- Deployment order: Lambda (no impact) → S3 lifecycle (no impact) → RDS during maintenance window
- Stack policies: prevent accidental replacement of critical resources. Add stack policy denying Replace action on AWS::RDS::DBInstance
- DeletionPolicy: set DeletionPolicy: Retain on RDS — if CloudFormation tries to delete it, it stays (backup for mistakes)
- Rollback triggers: add CloudWatch alarms as rollback triggers — if alarm fires during deploy, auto-rollback
- Multi-stack: separate RDS into its own stack — changes to application stack don't risk database
- Parameter Store / Secrets Manager: don't put RDS credentials in template parameters — use dynamic references `{{resolve:secretsmanager:...}}`

Follow-up: What's the difference between a CloudFormation rollback and a stack update with a previous Change Set? In what scenarios does CloudFormation rollback fail, and how do you recover a `ROLLBACK_FAILED` stack?

Q11
9

CDK — Production Best Practices

Medium

Your team is adopting AWS CDK. A junior engineer is putting all infrastructure (VPC, EKS, RDS, Lambda, DynamoDB) into a single CDK App with one Stack. What problems will this cause and how do you reorganize it?

Testing: CDK stack organization, construct levels, env-specific configuration, CDK Pipelines

✓ Key Answer Points:

- Problem 1 — blast radius: one stack update = CloudFormation deploys everything. A Lambda code change risks accidentally modifying VPC or RDS
- Problem 2 — deploy time: large stacks take 30-60 min to deploy. Small stacks are faster and easier to debug
- Problem 3 — drift and state: CloudFormation tracks state per stack. One large stack = one large state file prone to drift
 - Recommended stack structure: NetworkStack (VPC, subnets, SGs), DataStack (RDS, DynamoDB, depends on NetworkStack), ComputeStack (EKS, Lambda, depends on NetworkStack + DataStack), ApplicationStack (application-level resources)
 - Cross-stack references: export from one stack, import via `Fn.importValue()` or CDK's cross-stack construct references
 - CDK Environments: separate env per account/region. `NetworkStack.env = {account: '123456789', region: 'us-`

```
east-1}
```

- CDK Aspects: apply compliance checks across all constructs (e.g., ensure all S3 buckets have versioning)
- CDK Pipelines: self-mutating pipeline — code change → pipeline updates itself → deploys to all stages
- L1 vs L2 vs L3 constructs: L1 = CloudFormation resource (CfnBucket), L2 = opinionated (Bucket), L3 = pattern (ApplicationLoadBalancedFargateService). Use L2+ for safety defaults
- CDK context: store environment-specific values in cdk.context.json — VPC IDs, account IDs, etc.

 **Follow-up:** How does CDK handle circular dependencies between stacks? What do you do when Stack A needs an output from Stack B and Stack B also needs an output from Stack A?

Q12
0

CloudFormation — StackSets & Multi-Account

Medium

Your organization has 50 AWS accounts across 4 organizational units. You need to deploy a baseline security configuration (CloudTrail, Config, GuardDuty, VPC Flow Logs) to ALL accounts. Manual deployment to each account is not feasible. Design the solution.

 **Testing:** StackSets, AWS Organizations integration, service-managed StackSets, delegated administrator

✓ Key Answer Points:

- CloudFormation StackSets: deploy one stack template to multiple accounts and/or regions from a central management account
- Service-managed StackSets (preferred): integrate with AWS Organizations. Automatically deploy to new accounts when they join an OU
- Self-managed StackSets: requires creating IAM roles in each target account manually. More setup, more control
- Automatic deployment: set deployment options: automatic deployment to OUs, retain stacks when accounts removed = false (clean up on account removal)
- Deployment order: deploy to sandbox OU first → production OU second. Use failure tolerance to avoid blocking all accounts if one fails
- Failure tolerance: set failureToleranceCount: 5 — allow up to 5 account failures before aborting (for large rollouts)
- Max concurrent accounts: control how many accounts deploy simultaneously to avoid API throttling
- Delegated admin: Security OU account (not org management account) can be delegated admin for StackSets — better security posture
- Drift detection: run drift detection on all StackSet instances periodically — manual changes in accounts will show as drifted
- SCPs complement StackSets: SCP prevents disabling GuardDuty (compliance), StackSet deploys it (provisioning)

 **Follow-up:** What happens to resources deployed by a StackSet when an account leaves the AWS Organization? What if the account re-joins later?



Cost Optimisation

Reserved Instances · Savings Plans · Spot · Right-Sizing · Tagging

Q12
1

Cost — Reserved Instances Strategy

Medium

Your monthly AWS bill is \$150,000. EC2 is \$80,000, RDS is \$30,000, and the rest is various services. You're running production 24/7 with predictable load. How do you reduce costs without affecting reliability?

Testing: Reserved Instances, Savings Plans, Compute Optimizer, commitment strategy

Key Answer Points:

- Savings Plans (preferred over RIs for EC2): Compute Savings Plans apply to EC2, Lambda, Fargate regardless of instance type/size/region. 1-year no-upfront = 17% savings. 3-year all-upfront = 40%+ savings
- EC2 Instance Savings Plans: apply to specific instance family in region — higher savings (up to 45%) but less flexible
- RDS Reserved Instances: 1-year no-upfront saves ~30%; 3-year all-upfront saves ~50%+. Must match DB engine, instance class, Multi-AZ
- Coverage target: aim for 70% of base load covered by Savings Plans/RIs. Keep 30% on-demand for flexibility
- AWS Compute Optimizer: analyzes CloudWatch metrics, recommends right-sized instance types. Often finds 20-30% savings from right-sizing alone
- Identify waste: AWS Cost Anomaly Detection + Cost Explorer → find idle resources (EC2 instances < 5% CPU), unattached EBS volumes, old snapshots, unused Elastic IPs
- Tagging: enforce cost allocation tags (team, environment, project) via SCPs — required for granular cost analysis
- S3 Intelligent-Tiering: automatically moves objects — saves 20-40% on object storage at no management cost
- Graviton instances: switch to ARM-based Graviton3 — 40% better price-performance for compute
- Trusted Advisor: checks for underutilized resources, idle load balancers, unassociated Elastic IPs, low-utilization EC2

Follow-up: What's the difference between Savings Plans and Reserved Instances? Under what circumstances would you choose RIs over Savings Plans for EC2?

Q12
2

Cost — Spot Instance Architecture

Hard

Your data processing pipeline runs 200 r5.2xlarge instances for 8 hours every night. Cost is \$800/night on-demand. You want to use Spot to reduce cost but can't afford to lose processing in flight. Design the architecture.

Testing: Spot instance interruption handling, checkpointing, diversification, Spot Fleet / EC2 Fleet

Key Answer Points:

- Spot savings: r5.2xlarge Spot typically 60-70% cheaper than on-demand = ~\$500-560/night savings
- Spot interruption: 2-minute warning before reclamation. Must checkpoint state every 1-2 minutes
- Diversification: request multiple instance types (r5.2xlarge, r4.2xlarge, r5a.2xlarge, m5.4xlarge) across AZs. More pools = lower interruption probability
- EC2 Auto Scaling with Spot: use capacity-optimized allocation strategy — selects pools with lowest interruption probability
- AWS Batch: managed service specifically for batch workloads on Spot. Handles retries, checkpointing lifecycle, instance replacement automatically
- Checkpointing: save processing state to S3 every N minutes. On interruption: new instance reads checkpoint and resumes
- Spot + On-Demand base: run 20% On-Demand base capacity for guaranteed minimum, 80% Spot. On mass

interruption, pipeline slows but doesn't stop

- Spot interruption notice handler: CloudWatch Events rule → Lambda on EC2 Spot Instance Interruption Notice → checkpoint current state, drain queue
- SQS-based work queue: each processing unit reads from SQS. On interruption: message becomes visible again after visibilityTimeout → another instance picks it up
- Cost estimate: 200 instances × 8h × \$0.32 (Spot r5.2xlarge) vs \$1.008 (On-Demand) = ~\$256 vs \$806 per night

 **Follow-up:** How does EC2 Spot Placement Score help you plan Spot workloads? What does it tell you and how would you use it before a large Spot launch?

Q12
3

Cost — Tagging & Showback

Medium

Engineering teams are using AWS resources but have no visibility into what they're spending. The CFO asks for per-team, per-environment cost breakdown. You have no current tagging standard. How do you implement this?

 **Testing:** Cost allocation tags, tag policies, SCPs, AWS Cost Explorer, chargeback

✓ Key Answer Points:

- Step 1: define tagging standard. Mandatory tags: Team (platform/frontend/backend/data), Environment (prod/staging/dev), Project (project name), CostCenter (finance department code)
- Step 2: enforce via AWS Organizations Tag Policies — validates tag values on resource creation, generates noncompliance reports
- Step 3: SCPs to enforce tagging — deny EC2/RDS/EKS creation without required tags
- Step 4: activate cost allocation tags in Billing console — tags must be activated before they appear in Cost Explorer
- Step 5: tag existing resources — use Tag Editor to find and tag all untagged resources across services
- AWS Config Rule: required-tags rule — detects resources created without mandatory tags. Auto-remediation: Lambda adds default 'untagged' tag and creates Jira ticket
- Cost Explorer: filter by tag key/value to see per-team spend. Export to S3 daily via Cost and Usage Report (CUR)
- CUR + Athena + QuickSight: build custom cost dashboard. SQL on CUR data is more flexible than Cost Explorer
- Showback vs Chargeback: Showback = show teams what they spent (visibility). Chargeback = actually bill the team's budget. Start with showback.
- Shared resources (EKS cluster, TGW): allocate proportionally by pod count, CPU, or custom split

 **Follow-up:** How do you handle cost allocation for shared infrastructure (EKS cluster used by 5 teams, centralized Transit Gateway, shared NAT Gateway)? These don't cleanly map to one team's tag.



Observability

CloudWatch · X-Ray · CloudTrail · AWS Distro for OpenTelemetry

Q12
4

CloudWatch — Logs Architecture

Medium

Your application generates 50 GB of logs per day across 200 ECS tasks. The team uses CloudWatch Logs but costs are \$3,000/month just for log storage. They can't find relevant logs quickly during incidents. Design a better logs architecture.

Testing: CloudWatch Logs retention, log groups, insights, subscription filters, cost optimisation

✓ Key Answer Points:

- Cost breakdown: CloudWatch Logs costs \$0.50/GB ingestion + \$0.03/GB-month storage. 50GB/day × 30 days × \$0.50 = \$750 ingestion + 1,500 GB × \$0.03 = \$45 storage. But: if retaining 6 months = 9,000 GB × \$0.03 = \$270/month. Estimate doesn't match — check actual retention settings
- Fix 1: set log retention policy. Prod: 30-90 days in CloudWatch. Dev/staging: 7 days. Archive to S3 (Glacier) for long-term
- Fix 2: subscription filter → Kinesis Data Firehose → S3 (compressed Parquet) → Athena for historical analysis. S3 storage is 100x cheaper than CloudWatch
- Fix 3: Metric filters — extract key metrics (error count, latency) from logs into CloudWatch Metrics. Alert on metrics, not log scans
- CloudWatch Logs Insights: ad-hoc query language for log analysis. Much faster than downloading and grepping
- Structured logging (JSON): format logs as JSON. Insights queries become 20x faster and more powerful
- Log group per service: separate log groups per application. /ecs/myapp-prod, /ecs/myapp-staging. Apply different retention per environment
- Contributor Insights: find top error messages, top slow endpoints, top failing users automatically
- Alarm on log patterns: CloudWatch Metric Filter → Alarm on ERROR count spike
- EMF (Embedded Metric Format): emit structured metrics from application logs — bypass custom metrics API cost

Follow-up: How does CloudWatch Logs Insights pricing work, and at what scale would you switch from Logs Insights to Athena-on-S3 for cost efficiency?

Q12
5

X-Ray — Distributed Tracing

Hard

A user reports that some API requests take 8-10 seconds but you can't reproduce it. The request goes through: API Gateway → Lambda → 3 downstream microservices → DynamoDB. How do you use X-Ray to find the bottleneck?

Testing: X-Ray service map, trace sampling, segments/subsegments, annotations, trace ID propagation

✓ Key Answer Points:

- X-Ray Service Map: shows all services and their connections with latency percentiles (p50, p90, p99). The slow node will show elevated p99 in the service map
- Trace filtering: in X-Ray console, filter by responseTime > 5 → shows only slow traces. Look at complete trace waterfall
- Waterfall view: each trace shows all segments and subsegments with timing. The gap between parent segment end and child start = latency to call downstream service
- Find the bottleneck: look for which segment has the largest duration. If Lambda → Service2 shows 7s gap: Service2 is the culprit
- X-Ray Groups: create a group for high-latency requests: filter expression 'responsetime > 5'. Separate analysis for slow requests
- Annotations vs Metadata: Annotations are indexed and filterable (use for userId, requestId, tenantId).

Metadata is stored but not indexed (use for full request/response bodies)

- Add custom annotations: `xray.captureFunc('getUserProfile', () => { ... AWSXRay.resolveSegment().addAnnotation('userId', userId); })`
- DynamoDB: X-Ray auto-instruments DynamoDB SDK calls. Subsegment shows table name, operation, RCU consumed — check for hot partitions
- Sampling: default 1 req/sec + 5% → increase sampling for specific endpoints: add sampling rule for `path='/api/slow'` at 100%
- X-Ray + CloudWatch ServiceLens: unified view of service health combining metrics, logs, and traces

 **Follow-up:** How does X-Ray handle trace context propagation through SQS? If Lambda A sends to SQS and Lambda B consumes, does the trace continue? What are the limitations?

Q12
6

CloudWatch — Alarms & Composite Alarms

Medium

Your on-call team is overwhelmed with alerts. During a deployment, they receive 40 separate CloudWatch alarms firing simultaneously (CPU, memory, error rate, latency — all for the same cause: a bad deploy). How do you redesign the alerting to reduce noise while keeping coverage?

 **Testing:** Composite alarms, alarm hierarchies, anomaly detection, alarm routing

✓ Key Answer Points:

- Composite Alarms: create a parent alarm that fires ONLY when multiple child alarms are simultaneously in ALARM state
- Example: `DeploymentHealthAlarm = CPU > 80% AND ErrorRate > 5% AND Latency > 2s`. Single notification instead of 3
- Alarm hierarchy: Service-level composite alarm (fires to PagerDuty) aggregates symptom alarms (fire to Slack/low-noise channel only)
- Anomaly Detection: instead of static thresholds, use ML-based anomaly detection — adapts to traffic patterns
- During deploy: suppress expected alarms using alarm state suppressor. Set parent alarm to OK during deployment window
- Severity routing: p1 alarms (service-level composite) → PagerDuty. p2 alarms (individual metrics) → Slack only
- Missing data treatment: `INSUFFICIENT_DATA` → ALARM vs BREACHING vs NOT_BREACHING. Set appropriately — missing data should not auto-alarm if it means service restarted
- For: set period and evaluation periods to avoid transient spikes. CPU > 80% for 3 consecutive 1-minute periods, not instantaneous
- Alarm description: every alarm must have runbook URL in AlarmDescription field — on-call knows what to do
- Metric math alarms: alarm on derived metrics. Error rate = errors/total. Single alarm on the ratio, not two separate alarms

 **Follow-up:** How do CloudWatch Composite Alarms handle the scenario where one child alarm is in `INSUFFICIENT_DATA` state? Does the composite alarm fire, and can you configure this behaviour?



Secrets & Configuration Management

Secrets Manager · Parameter Store · KMS · Rotation

Q12
7

Secrets Manager — Rotation Architecture

Hard

You have 200 microservices each connecting to different databases. Credentials are currently hardcoded in environment variables (set during deployment). A security audit requires automatic credential rotation every 30 days with zero application downtime. Design the rotation architecture.

Testing: Secrets Manager rotation, multi-user rotation strategy, application integration, zero-downtime rotation

Key Answer Points:

- Secrets Manager: stores credentials, auto-rotates on schedule using a Lambda function
- Single-user rotation: Secrets Manager updates the DB password and the secret simultaneously. Problem: brief window where old password invalid, new secret not yet fetched by apps
 - Multi-user rotation (zero-downtime): maintain two DB users (myapp_a, myapp_b). Alternate between them on rotation. App uses whichever is current — no gap
- Rotation Lambda: built-in Lambda functions for RDS (MySQL, Postgres, Oracle) provided by Secrets Manager. Deploy once, reuse across all 200 services
- Application change: replace hardcoded env vars with Secrets Manager API call at startup (or periodically). aws-secretsmanager-caching client caches the secret and refreshes before expiry
- IRSA for EKS pods: pod has IAM role granting secretsmanager:GetSecretValue on specific ARNs only
- AWS Secrets Manager Agent: sidecar that fetches secrets and exposes via local HTTP endpoint — app calls localhost, not AWS API directly. Reduces IAM calls
- Fallback: if Secrets Manager unreachable, use last cached value (caching client does this automatically)
- Monitoring: CloudWatch metric: GetSecretValue by SecretId. Alert on rotation failures: SecretRotationFailed event in EventBridge
- Migration path: use Secrets Manager CSI Driver (Kubernetes) — mounts secrets as files; apps read file instead of API

Follow-up: What happens to an application's database connection pool during a multi-user rotation? Existing connections use the old user — do they break? When do they get the new credentials?

Q12
8

Parameter Store — Configuration Architecture

Medium

You need to manage application configuration across 4 environments (dev/staging/staging2/prod). Different environments have different values for 50+ parameters (database URLs, feature flags, API endpoints). Some values are sensitive (API keys), some are not. Design the configuration management system.

Testing: Parameter Store hierarchy, SecureString vs String, KMS, cross-account, versioning

Key Answer Points:

- Parameter Store hierarchy: use `/env/{service}/{parameter}` naming convention
- Example: `/prod/myapp/DATABASE_URL`, `/staging/myapp/DATABASE_URL`, `/dev/myapp/DATABASE_URL`
- SecureString (KMS-encrypted): API keys, passwords, tokens. GetParameter with `WithDecryption=true`
- String type: non-sensitive config (feature flags, URLs, timeouts). GetParameter without decryption
- StringList: comma-separated values (allow-listed IPs, multiple endpoints)
- Application integration: at startup, fetch all parameters under `/prod/myapp/` using `GetParametersByPath` with `recursive=true`. Single API call for all config
 - AWS AppConfig: built on Parameter Store, adds deployment strategies (canary, linear), rollback, validation — use for feature flags and config that needs controlled rollout
 - KMS per environment: separate KMS key per environment. Dev team cannot decrypt prod parameters even if

they can read them

- Parameter versions: Parameter Store automatically versions parameters. Roll back with `GetParameter(version=N)`
- EKS integration: AWS Secrets Store CSI Driver mounts Parameter Store values as Kubernetes secrets / volumes
- Cross-account: use resource policy on KMS key to allow cross-account decryption for shared parameters

 **Follow-up:** *How does Parameter Store handle the case where an application reads a parameter, then the parameter is updated, and the application is still running? How do you trigger a config reload?*



CI/CD on AWS

CodePipeline · CodeBuild · CodeDeploy · ECR · GitOps on AWS

Q12
9

CodePipeline — Multi-Stage Pipeline Design

Medium

Design a CI/CD pipeline for a microservice that: runs on ECS Fargate, has 3 environments (dev/staging/prod), needs automated tests, security scanning, manual approval for prod, and blue/green deployment to prod.

Testing: CodePipeline stages, CodeBuild projects, approval actions, CodeDeploy blue/green, ECS integration

Key Answer Points:

- Pipeline stages: Source → Build → Test → Security → Deploy-Dev → Deploy-Staging → Approval → Deploy-Prod
- Source stage: CodeCommit or GitHub (via CodeStar connection). Triggers on push to main branch
- Build stage: CodeBuild — runs 'docker build', pushes image to ECR with tag = commit SHA
- Test stage: CodeBuild — runs unit tests, integration tests against ephemeral environment
- Security stage: CodeBuild — runs ECR image scan (Trivy or Inspector), SAST (Checkov for IaC, Semgrep for code), fails pipeline if critical CVEs found
- Deploy-Dev: ECS Deploy action (in-place rolling). Dev = fast feedback, downtime acceptable
- Deploy-Staging: ECS Deploy action (in-place rolling). Run E2E tests as a separate CodeBuild action after deploy
- Manual Approval: SNS notification to engineering lead. Approval URL in notification. 7-day expiry
- Deploy-Prod: CodeDeploy blue/green. Creates new task set, shifts traffic via ALB, monitors during bake period, terminates old task set
- Notifications: EventBridge rule on pipeline state changes → SNS → Slack via Lambda
- Artifacts: CodePipeline passes image URI between stages via artifact store (S3 bucket).
imageDefinitions.json: [{name: 'container', imageUri: 'ecr.../myapp:abc123'}]

Follow-up: How does CodeDeploy blue/green deployment for ECS handle a case where the green task set passes health checks but then starts returning errors after 5 minutes (after the bake period)? What are your options?

Q13
0

CodeBuild — Security & Performance

Hard

Your CodeBuild builds are taking 12 minutes each (90% of which is downloading dependencies). The build environment uses a default public CodeBuild image. Security team flags: builds have access to production IAM role and are pulling from public DockerHub. Fix both issues.

Testing: CodeBuild caching, private build environments, VPC integration, least-privilege IAM

Key Answer Points:

- Performance — dependency caching: enable CodeBuild local cache (S3 cache) for node_modules, .m2, .gradle, pip packages
- buildspec.yml cache section: cache paths: [/root/.m2/**/*, 'node_modules/**/*']
- S3 cache: CodeBuild uploads/downloads cache to S3. Reduces npm install from 4 min to 30 sec
- Custom Docker image with pre-installed dependencies: push base image to ECR with dependencies baked in. CodeBuild pulls from ECR = faster than installing each time
- Security — CodeBuild IAM role: scope to least privilege. Only needs: ECR:GetAuthorizationToken, ECR:BatchCheckLayerAvailability, ECR:PutImage — NOT production DB access
- Never give CodeBuild production IAM permissions directly. Use IAM role assumption: CodeBuild assumes a specific role per environment only during that stage
- DockerHub rate limiting: CodeBuild pulls from DockerHub = rate limited at 100/6h on shared IPs. Fix: mirror

base images to ECR

- ECR public gallery: use ECR Public (gallery.ecr.aws) as a rate-limit-free alternative for community images
- VPC CodeBuild: for accessing private resources (RDS, internal APIs in tests). CodeBuild deploys into your VPC via ENI
- Secrets: CodeBuild env vars can reference Parameter Store/Secrets Manager. NEVER put secrets as plaintext CodeBuild env vars
- Build badge + notifications: CodeBuild integrates with GitHub status checks — shows build status on PR

 **Follow-up:** If CodeBuild runs in your VPC for integration tests, what NAT Gateway configuration is required for CodeBuild to also pull from ECR and access the internet for external dependencies?

Q13
1

GitOps on AWS — ArgoCD + ECR + CodeBuild

Hard

Your team uses ArgoCD on EKS for GitOps-based deployments. The CI pipeline (CodeBuild) builds and pushes images to ECR. But there's a gap: how does ArgoCD know to deploy the new image? The git repo doesn't get updated automatically. Design the full GitOps pipeline.

 **Testing:** GitOps workflow, image updater, ECR event-driven, git automation, separation of CI and CD

✓ Key Answer Points:

- Problem: GitOps requires git as source of truth. CI pushing to ECR doesn't update git → ArgoCD doesn't deploy
- Solution A: ArgoCD Image Updater — watches ECR for new image tags, automatically commits updated image tag to git, ArgoCD syncs the commit
- Image Updater setup: deploy in ArgoCD namespace, configure ECR registry access via IRSA, annotate Application with update policy
- Update strategies: digest (always latest), semver (update if newer semver tag), latest (latest tag — not recommended)
- Solution B: CodeBuild post-push step — after ECR push, CodeBuild clones the infra repo, updates the image tag in kustomization.yaml or Helm values, creates a PR (or direct commit to env branch)
- Branch-per-environment: CodeBuild updates dev branch → ArgoCD-dev syncs. After testing, PR to staging branch, etc.
- ECR push event → EventBridge → Lambda → update git → ArgoCD detects → sync
- Immutable tags: use commit SHA as image tag (not 'latest'). ECR tag immutability prevents overwriting
- Code repo vs config repo: keep application code and K8s manifests in separate repos. CI owns code repo. CD (ArgoCD) owns config repo
- Audit trail: every deployment is a git commit in the config repo — full history of what was deployed when and by whom (CI system)

 **Follow-up:** What are the security implications of giving ArgoCD Image Updater write access to your git repository? How do you scope this access minimally using GitHub fine-grained tokens or GitLab deploy keys?



Advanced Architecture Scenarios

Multi-Service Design · Resilience · Real-World Incidents

Q13
2

Resilience — Multi-Region Active-Active

Hard

Design a multi-region active-active architecture for a high-traffic e-commerce platform. Both us-east-1 and eu-west-1 must serve traffic simultaneously. Users in Europe should go to eu-west-1. Orders must not be lost if one region fails.

Testing: Route 53 geolocation, Global Accelerator, DynamoDB Global Tables, active-active design patterns

✓ Key Answer Points:

- Traffic routing: Route 53 Geolocation Routing — EU traffic → eu-west-1 ALB, US traffic → us-east-1 ALB. With health check: if eu-west-1 fails, reroute EU traffic to us-east-1
- AWS Global Accelerator: anycast IPs, routes users to nearest healthy region. Faster failover than DNS (no TTL wait) and better path optimization than Route 53 alone
- Database: DynamoDB Global Tables — multi-active, sub-second replication. Write locally, read locally. Last-writer-wins conflict resolution
- Order deduplication: use DynamoDB conditional write (PK = orderId) — only succeeds if orderId doesn't exist. Prevents duplicate orders even if request reaches both regions (e.g., retries)
- Session: ElastiCache Global Datastore — active-active Redis across regions. User session survives regional failover
- Images/assets: S3 Cross-Region Replication — eu-west-1 S3 bucket replicates to us-east-1. CloudFront with multiple origins
- Application: EKS in both regions, identical Helm chart versions. Deploy simultaneously via GitOps
- Inventory consistency: DynamoDB strongly consistent reads for inventory reservation. Conditional writes for stock decrement
- Monitoring: Route 53 health checks for automated failover. CloudWatch cross-region dashboard
- RPO = near-zero (DynamoDB Global Tables < 1s replication). RTO = < 60s (Route 53 health check failover + DNS TTL)

Follow-up: *DynamoDB Global Tables uses last-writer-wins for conflict resolution. In an e-commerce scenario, two users simultaneously purchase the last item in stock — one request reaches us-east-1, one reaches eu-west-1. Walk through exactly what happens and whether overselling is possible.*

Q13
3

Incident — Lambda Throttling Cascade

Hard

Production incident: a marketing email campaign was sent to 5 million users. Each email has a tracking pixel that calls an API Gateway → Lambda. The Lambda suddenly has 50,000 concurrent executions, hits the account limit (1,000), and now OTHER critical Lambda functions (payment processing) are being throttled. Immediate response and prevention?

Testing: Lambda concurrency, account vs function limits, reserved concurrency, SQS buffering, throttle handling

✓ Key Answer Points:

- Immediate: set reserved concurrency on the email tracking Lambda to 100. This caps it and frees quota for payment Lambda
- Immediate 2: set reserved concurrency on payment Lambda to 200 — guarantees it gets quota regardless of other Lambdas
- Reserved concurrency = reservation from the shared pool. If email tracking gets 100 reserved, payment gets 200 reserved, only 700 is shared for others
- Root cause: Lambda has a soft limit of 1,000 concurrent executions per account per region (default). All Lambdas share this pool
- Prevention architecture: API Gateway → SQS → Lambda (not API Gateway → Lambda directly). SQS buffers

5M pixel requests. Lambda processes at a controlled rate. No burst problem

- With SQS: Lambda scales based on SQS queue depth, not request rate. 1,000 max concurrent × batch size 10 = 10,000 messages/second processing. Queue depth grows but processes over time
- Request quota increase: submit service quota increase for concurrent Lambda executions. 3,000 burst limit in us-east-1 and EU, 500 in other regions
- Provisioned Concurrency: pre-warm payment Lambda to prevent cold start + guarantee capacity
- Monitoring: CloudWatch Lambda Throttles metric per function. Alert: Throttles > 0 for any critical Lambda
- Account-level view: CloudWatch namespace AWS/Lambda, metric ConcurrentExecutions across all functions — near-limit alert

 **Follow-up:** How does Lambda Powertools for Python/Node.js help with building resilient Lambdas, specifically around idempotency, structured logging, and tracing? What's the implementation overhead?

Q13
4

Architecture — SQS vs Kinesis Decision

Medium

You need to stream application events for real-time analytics. You're choosing between SQS, Kinesis Data Streams, and Kinesis Firehose. Events are: user clickstream (1,000/sec), order events (50/sec), inventory updates (200/sec). Different consumers: analytics team, ML team, fraud team. Walk through your decision.

 **Testing:** SQS vs Kinesis architectural differences, replay, fan-out, retention, ordering

✓ Key Answer Points:

- SQS: message queue. Consumer removes the message after processing. No replay. Multiple consumers = fan-out via SNS or duplicate queues
- Kinesis Data Streams: log stream. Consumer reads at its own pace. Message stays for 24h (up to 365 days). Multiple consumers read independently from the same stream
- Key difference: Kinesis supports replay (rewind and reprocess). SQS messages are deleted after consumption
- Clickstream (1,000/sec, multiple consumers): Kinesis Data Streams. Analytics, ML, and fraud teams each read independently. They can rewind and replay if a new ML model needs historical data
- Order events (50/sec, needs ordering per order ID): Kinesis (partition key = orderId) — events for same order go to same shard, in order. SQS FIFO only guarantees order within a MessageGroup
- Inventory updates (200/sec, single consumer, no replay needed): SQS Standard is fine. Simpler, cheaper
- Kinesis Firehose: delivery service, NOT for real-time processing. Kinesis Data Streams → real-time processing. Firehose → batch delivery to S3/Redshift/ES (buffer 60s-15min)
- Shard capacity: each Kinesis shard handles 1MB/s write, 2MB/s read, 1,000 records/s. For 1,000 events/sec: need at least 1 shard (assuming small events). Plan for 2x headroom
- MSK (Managed Kafka): for highest throughput, long retention, rich consumer group semantics — heavier weight than Kinesis
- Cost: SQS per-message pricing (\$0.40/million). Kinesis per-shard-hour (\$0.015/shard-hour). At low volume: SQS cheaper. At high volume with replay: Kinesis cheaper

 **Follow-up:** How does Kinesis Data Streams handle a slow consumer? If the analytics team's Lambda is slow and falls behind, what happens to the data? What happens after 24 hours of retention?

Q13
5

Security — Data Exfiltration Prevention

Hard

Your security team is concerned about data exfiltration from EC2 instances that have access to S3 buckets with sensitive customer data. An insider threat or compromised instance could exfiltrate data. What AWS controls do you implement to detect and prevent this?

 **Testing:** S3 bucket policies, VPC endpoints, GuardDuty, Macie, CloudTrail, network controls for data loss prevention

✓ Key Answer Points:

- Control 1: S3 VPC Endpoint (Gateway) — add condition to bucket policy: deny GetObject unless request comes via VPC endpoint. Instances outside the VPC cannot access the bucket even with valid credentials
- Control 2: S3 bucket policy — deny GetObject if request comes from unexpected principal (not the expected EC2 role ARN) or unexpected network (not the VPC endpoint)
- Control 3: restrict EC2 internet access — place sensitive EC2 instances in private subnets with no NAT Gateway. Can access S3 via endpoint but cannot exfiltrate to internet
- Control 4: VPC Flow Logs + CloudTrail Data Events — enable CloudTrail S3 data events (GetObject, PutObject). Creates audit trail for every S3 access
- Control 5: Amazon GuardDuty — detects anomalous S3 access patterns (unusual volume, access from unexpected location, credential exfiltration)
- Control 6: Amazon Macie — discovers and classifies sensitive data in S3 (PII, credit card numbers). Alerts when sensitive data is accessed unexpectedly
- Control 7: S3 Object Lock (WORM) — prevents deletion of objects during retention period. Compliance with regulatory requirements
- Control 8: bucket policy deny public access — enable S3 Block Public Access at account and bucket level
- Detect: CloudTrail → S3 events to CloudWatch Logs → Metric Filter on high-volume GetObject from single IAM principal → Alarm → SOC notification
- Response: IAM policy deny (disable the role) via automated Lambda triggered by GuardDuty finding

 **Follow-up:** How do you balance data exfiltration controls with operational access requirements? If a legitimate ML training job needs to read 50 TB from S3 rapidly, how do you design controls that allow this while still blocking exfiltration?

Q13
6

Performance — API Gateway Optimization

Medium

Your REST API built on API Gateway + Lambda has p99 latency of 3 seconds. The average Lambda execution time is 200ms. Where are the other 2.8 seconds going, and how do you fix it?

 **Testing:** API Gateway latency breakdown, Lambda cold starts, caching, connection reuse, HTTP APIs vs REST APIs

✓ Key Answer Points:

- API Gateway overhead itself: REST API adds 5-25ms typically. Check API Gateway Integration Latency vs Total Latency in CloudWatch to isolate APIGW vs Lambda
- Lambda cold start: if 2.8s spike is intermittent (on first request after idle), it's Lambda cold start. Java/Python/Node cold starts: 100ms-2s depending on package size
- Cold start fix: Provisioned Concurrency pre-warms Lambda instances. Or: Lambda SnapStart (Java only) — restores from snapshot
- Package size: each MB of deployment package adds ~10ms cold start. Use Lambda Layers for shared deps. Use tree-shaking to reduce bundle size
- X-Ray: enable X-Ray on API Gateway + Lambda. Trace waterfall shows exactly which component takes the time
- HTTP API vs REST API: HTTP API is 60% cheaper and 60% faster (lower overhead). If you don't need REST API features (usage plans, request validation, API keys) — switch to HTTP API
- API Gateway caching: enable response caching (0.5-237 GB). Cache TTL: 300s default. Cached responses return in <5ms. Enable for GET endpoints with stable responses
- Lambda timeout vs actual duration: if Lambda has a 30s timeout but p99 is 3s, check for: DB connection establishment overhead, external API calls, N+1 queries
- Connection reuse: Lambda must reuse DB connections across invocations. Initialize DB client outside handler function. RDS Proxy for connection pooling
- VPC Lambda: adds ~300ms-1s first call due to ENI allocation. Use Hyperplane ENIs (transparent to you but check ENI creation events)

 **Follow-up:** API Gateway has a hard maximum integration timeout of 29 seconds. If your Lambda sometimes takes 35 seconds (long-running report generation), what architecture change do you make?

Q13
7**Networking — Transit Gateway Advanced****Hard**

You have 20 VPCs: 15 application VPCs, 2 shared services VPCs (DNS, Monitoring), 1 inspection VPC (with firewall), 1 egress VPC (NAT), and 1 management VPC. You need: all application VPCs to route outbound internet through the egress VPC, all traffic to pass through the inspection VPC, application VPCs cannot communicate with each other directly. Design the TGW routing.

 **Testing:** TGW route tables, isolated routing domains, inspection with TGW, spoke-hub architecture

 Key Answer Points:

- TGW attachments: 20 VPC attachments + TGW itself = hub of all connectivity
- Route table 1 (Spoke RT — for 15 app VPCs): default route 0.0.0.0/0 → TGW attachment of inspection VPC. No routes to other spoke VPCs. Result: all traffic goes to inspection first
- Route table 2 (Inspection RT — for inspection VPC): routes for all app VPC CIDRs → respective attachments. Default route → egress VPC attachment
- Route table 3 (Egress RT — for egress VPC): routes back to app VPC CIDRs → inspection VPC attachment. Default route: NAT Gateway in this VPC
- Shared services RT: routes for shared services CIDR → shared services VPCs. App VPCs route to shared services through inspection? Or directly? Policy decision
- AWS Network Firewall in inspection VPC: stateful rules engine. App VPC traffic → GWLB Endpoint → Firewall → route back. Suricata-compatible rules
- Blackhole routes: in spoke RT, add blackhole routes for RFC1918 ranges that aren't used. Prevents lateral movement to unknown CIDRs
- TGW Route Table Association: each VPC attachment is ASSOCIATED with one RT (which RT does it use to look up routes?)
- TGW Route Table Propagation: each VPC attachment PROPAGATES its CIDR to specified RTs (which RTs learn about this VPC?)
- Result: app-to-app is impossible (spoke RT has no routes to other spokes). All internet traffic flows: App VPC → TGW → Inspection VPC → TGW → Egress VPC → Internet

 **Follow-up:** You add a new application VPC. What exact steps do you take to onboard it into this TGW architecture, and what mistakes would cause traffic to bypass the inspection VPC?

Q13
8**Compliance — CloudTrail & Audit Architecture****Medium**

Your organization needs to pass a SOC2 audit. The auditors require: 12 months of immutable audit logs for all API calls, evidence that logs haven't been tampered with, and the ability to answer 'who accessed what data when' for any resource. Design the audit logging architecture.

 **Testing:** CloudTrail organization trail, log file integrity, S3 object lock, Athena for queries

 Key Answer Points:

- Organization Trail: create one CloudTrail trail at the AWS Organizations management account level. Automatically captures all API calls in all member accounts
- Management events: capture all Read and Write management events. Include global service events (IAM, STS)
- Data events: capture S3 data events (GetObject, PutObject, DeleteObject) for sensitive S3 buckets. Capture Lambda function invocations for critical Lambdas
- S3 bucket with Object Lock (WORM): set trail to deliver to S3 bucket with Object Lock in COMPLIANCE mode (not governance — compliance cannot be overridden even by root). Retention: 12 months
- Log file validation: CloudTrail creates SHA-256 digest files for each log file and signs with KMS. `aws cloudtrail validate-logs` command verifies integrity. Tampered logs detected automatically
- Cross-account log bucket: trail delivers to a centralized security account S3 bucket. Application account roles cannot delete or modify logs in security account
- Queries: CloudTrail → S3 → AWS Athena. Create Athena table pointing to CloudTrail S3 prefix. Query: 'who called GetObject on S3 bucket X between date Y and Z?'

- Real-time analysis: CloudTrail → CloudWatch Logs → CloudWatch Insights for operational queries
- Security Hub: aggregates findings. CloudTrail findings (suspicious API calls) forwarded to Security Hub across all accounts
- Evidence collection for auditors: Athena query results + CloudTrail console S3 download. Digest file verification output = proof of integrity

 **Follow-up:** How does CloudTrail handle MFA-authenticated sessions vs assumed role sessions in the event records? If an engineer assumes a role to make an API call, does CloudTrail show the original IAM user's identity or the assumed role identity?

Q13
9

Disaster Recovery — RTO/RPO Design

Hard

Your application has an SLA of RTO=4 hours, RPO=1 hour for a full region failure. Current setup: EC2 + RDS + S3 in us-east-1. What DR strategy do you implement, and what would it cost vs a higher SLA of RTO=15 min, RPO=0?

 **Testing:** DR strategies: backup/restore, pilot light, warm standby, multi-region active-active; cost-benefit analysis

✓ Key Answer Points:

- RTO=4h, RPO=1h → Pilot Light strategy: minimal infrastructure running in dr-region (eu-west-1)
- Pilot Light components: RDS read replica in eu-west-1 (promoted to primary on failover), S3 CRR (Cross-Region Replication) for all buckets, AMLs copied to eu-west-1, minimal EC2 or ECS in stopped/scaled-to-zero state
- Recovery steps: promote RDS replica (2-5 min), scale up EC2/ECS in eu-west-1 (5-20 min), update Route 53 DNS to point to eu-west-1 ALB (TTL=60s propagation), total: 30-90 min — within 4h RTO
- RPO=1h: RDS read replica lag is typically <1 min. S3 CRR is typically <15min for most objects. Set RDS automated backup with 1-hour frequency as additional safety
- Cost: RDS read replica (~\$200-400/month depending on instance), S3 CRR storage + transfer fees (~\$50-100/month), minimal EC2 standby → total ~\$300-600/month additional
- RTO=15min, RPO=0 → Multi-region active-active: much higher cost and complexity
- Active-active components: DynamoDB Global Tables, Aurora Global Database, Global Accelerator, EKS in both regions
- Cost for active-active: 2× compute, DynamoDB Global Tables replication fee (\$0.175/million replicated writes), Aurora Global cross-region replication → easily 2-3× current infrastructure cost
- Runbook: document recovery steps. Test DR quarterly. Chaos engineering: actually failover to eu-west-1 in staging environment
- GameDay: schedule a quarterly 'region down' drill. Measure actual RTO vs target. Update runbook based on real measurements

 **Follow-up:** How does AWS Elastic Disaster Recovery (DRS) differ from manually managing pilot light? What are its advantages and at what point does its cost make it less attractive than manual pilot light?

Q14
0

Performance — Aurora vs RDS for MySQL

Medium

You're running MySQL on RDS (db.r5.2xlarge, Multi-AZ). Read replicas are struggling with lag during peak hours. A developer suggests migrating to Aurora MySQL. What are the actual architectural differences, and when does it make sense?

 **Testing:** Aurora cluster architecture, shared storage layer, Aurora Replicas vs RDS Read Replicas, failover speed

✓ Key Answer Points:

- RDS MySQL: traditional MySQL on managed EC2. Multi-AZ = synchronous replication to standby (via EBS). Read replicas use MySQL native replication (binlog) — can lag
- Aurora MySQL: shared distributed storage layer (6 copies across 3 AZs). Compute and storage separated.

Aurora Replicas read from the same storage — no replication lag

- Aurora Read Replica lag: near-zero (10-20ms) because they read from shared storage, not from binary log. RDS Read Replica lag: can be seconds to minutes under write load
- Aurora Failover: promotes a read replica to primary in ~30 seconds. RDS Multi-AZ failover: 60-120 seconds (DNS change + standby warm-up)
- Aurora storage: auto-scales 10 GB to 128 TB. You don't provision storage. RDS: you provision storage and manually extend
- Aurora Serverless v2: automatically scales compute (ACUs) based on load. Pay per ACU-hour. Good for variable workloads
- Aurora Global Database: cross-region replica with <1s lag. RDS cross-region replica: MySQL binlog = higher lag
- Cost: Aurora is typically 20-40% more expensive than RDS MySQL for same workload but you get: faster failover, near-zero replica lag, better scaling
- Migration: AWS DMS (Database Migration Service) or Aurora clone from RDS snapshot
- When NOT to use Aurora: very small databases (<10GB), Aurora pricing premium not justified, specific MySQL version compatibility requirements (Aurora lags MySQL version support by 6-12 months)

 **Follow-up:** Aurora stores 6 copies of data across 3 AZs, but only charges for 1 copy's storage. What happens during an AZ failure — how many copies remain and can Aurora still serve reads and writes?



Expert-Level Scenarios

Architecture decisions, trade-offs, and deep internals

Q14
1

EKS — Managed vs Self-Managed Node Groups

Medium

Your team is setting up EKS node groups. You're choosing between Managed Node Groups, Self-Managed Node Groups, and Fargate Profiles. Walk through the trade-offs and when you'd use each.

Testing: EKS node group types, control plane vs data plane, operational overhead, Karpenter

Key Answer Points:

- Managed Node Groups (MNG): AWS provisions and manages the EC2 instances, ASG, and node lifecycle. AMI updates (including security patches) handled by AWS rolling update. Simplest operational model
- MNG limitations: can't customize kubelet flags beyond supported parameters, can't use custom AMI easily, limited launch template customization
- Self-Managed Node Groups: you own the EC2 ASG, AMI, lifecycle. Full customization: custom AMI, custom kubelet flags, GPU drivers, EFA networking, custom labels/taints
- Self-managed cost: higher operational overhead. You manage AMI patching, node draining, ASG configuration
- Fargate Profiles: serverless pods. No nodes to manage. AWS handles capacity. Limitations: no DaemonSets, no privileged containers, no GPUs, no persistent volumes (only EFS), pod startup ~1 min
- Karpenter: next-generation node provisioner. Watches unschedulable pods and provisions exactly the right EC2 instance type in seconds. Consolidates nodes when underutilized. Much smarter than Cluster Autoscaler
- Karpenter vs MNG Cluster Autoscaler: Karpenter: directly calls EC2 API (faster), multi-instance-type, bin-packing. CA: works with existing ASG (slower, single instance type per group)
- Recommendation: MNG for most workloads (simplicity). Karpenter for cost optimization and mixed instance types. Fargate for small batch jobs. Self-managed only for GPU/EFA/special requirements
- EKS version drift: MNG can be N-2 behind control plane. Patch nodes before cluster upgrade

Follow-up: How does Karpenter's Consolidation feature work, and what disruption does it cause to running pods? When would you disable Consolidation in production?

Q14
2

Networking — IPv6 on AWS

Easy

Management asks whether you should enable IPv6 for the new VPC. What are the actual operational implications, benefits, and hidden complexity?

Testing: VPC IPv6 architecture, dual-stack, egress-only IGW, IPv6 cost model

Key Answer Points:

- IPv6 on AWS: VPC gets a /56 IPv6 CIDR from Amazon's pool (free). Each subnet gets a /64. EC2 instances get a global unicast IPv6 address
- No NAT for IPv6: IPv6 addresses are globally routable. Instances with IPv6 can reach the internet directly (with proper Security Group and route table rules)
- Egress-only Internet Gateway (EIGW): for private subnets — allows IPv6 egress (outbound) but blocks inbound. Equivalent of NAT Gateway for IPv6 (but free)
- Cost benefit: no NAT Gateway charges for IPv6 traffic. IPv6 traffic goes directly to/from instances. Can save significant NAT Gateway data processing fees
- Dual-stack: instances have both IPv4 and IPv6. Applications must listen on both protocols or use IPv6-only listener
- Security Group: must add IPv6 rules separately (:::0 for IPv6 equivalent of 0.0.0.0/0). Easy to forget and leave IPv6 uncontrolled
- DNS: IPv6 records are AAAA records. Application DNS lookups must handle both A and AAAA. Some older libraries default to IPv4 even when IPv6 is available
- EKS: enable IPv6 on EKS cluster → pods get IPv6 addresses, eliminating the IPv4 pod CIDR exhaustion

problem

- Not everything supports IPv6: some older AWS services, third-party SaaS endpoints may not have IPv6. Must maintain IPv4 as fallback
- Recommendation: enable IPv6 on new VPCs as best practice. Mainly beneficial for EKS (pod IP exhaustion) and reducing NAT costs

 **Follow-up:** How does AWS handle IPv6 address assignment to EC2 instances? Are they stable across stop/start? What changes in your application if you go IPv6-only?

Q14
3

Cost — Graviton Migration

Medium

Your workloads run on Intel x86 EC2 instances (m5, r5, c5). Graviton3 (m7g, r7g, c7g) promises 40% better price-performance. What's the actual migration process and what gotchas should you watch for?

 **Testing:** Graviton ARM architecture, cross-compilation, compatibility assessment, migration strategy

✓ Key Answer Points:

- Graviton is ARM64 architecture — software must be compiled for ARM64 (aarch64). This is the main migration concern
- Compatibility check: AWS Graviton Getting Started guide. Most languages work without code changes: Java (JDK 11+), Python, Node.js, Ruby, Go, Rust — all support ARM64
- Containers: Docker images must be multi-arch (or ARM64-specific). Rebuild: `docker buildx with --platform linux/arm64`. ECR supports multi-arch manifest lists
- Native code / compiled binaries: C/C++ code must be recompiled. Check third-party libraries for ARM64 support (most major libs now support it)
- Migration approach: (1) rebuild Docker images for arm64, (2) test on Graviton instance in staging, (3) run load test to validate performance, (4) switch production in canary fashion
- EKS: create new Graviton node group. Deploy arm64-compatible pods to it using `nodeSelector` or `nodeAffinity`. Run both x86 and arm64 node groups during migration
- Lambda: just change the architecture property to arm64. No other changes needed for Python/Node/Java. Cost: Graviton Lambda is 20% cheaper + 19% better performance = ~35% total improvement
- RDS: Aurora Graviton2 instances — same DB, just change instance class. No data migration
- Performance validation: not all workloads improve 40%. Single-threaded workloads may see less gain. Memory-intensive may see different profile. Always benchmark your specific workload
- Savings: m7g.large vs m5.large: ~15% cheaper + better performance. Most organizations see 20-30% cost reduction on Graviton migration

 **Follow-up:** Your application uses a Python library that wraps a native C extension (.so file). The C extension was compiled for x86. What are your options for running this on Graviton, and what's the fastest path to production?

Q14
4

IAM — Cross-Account Role Architecture

Hard

You have a central tooling account (CI/CD, monitoring) that needs to deploy to and read from 50 application accounts. You also have a security account that needs read-only access to audit logs in all accounts. Design the cross-account IAM architecture.

 **Testing:** Cross-account assume role, trust policies, permission boundaries, SCPs, hub-spoke IAM patterns

✓ Key Answer Points:

- Hub account pattern: central tooling account has IAM roles for CI/CD and monitoring systems
- Spoke accounts: each has a role that the hub account can assume. Trust policy: Principal = hub account CI/CD role ARN
- Cross-account deployment role: `DeployRole` in each spoke account. Trust: ARN of `CodeBuild` role in tooling

account. Permissions: CloudFormation, ECS, Lambda deploy permissions in that account

- Assume role chain: CodeBuild (in tooling account) → assume DeployRole in app-account-X → deploy resources
- Naming convention: all cross-account roles named consistently (e.g., CICDDeployRole). Tooling account knows to assume `arn:aws:iam::ACCOUNT_ID:role/CICDDeployRole`
- Security account read-only: SecurityAuditRole in every account. Trust: security account audit system. Permission: SecurityAudit managed policy (read all resources) + GetObject on CloudTrail S3
- StackSets: deploy the cross-account roles themselves via CloudFormation StackSets from the org management account
- Permission boundaries: on the DeployRole in spoke accounts, attach a permission boundary that prevents it from escalating privileges (can't create roles with more permissions than the boundary)
- SCPs: deny modification of DeployRole and SecurityAuditRole in spoke accounts — prevents spoke account admins from breaking the cross-account trust
- Monitoring: CloudTrail in each spoke account captures assume role events from tooling account — full audit of cross-account activity

 **Follow-up:** How do you handle the scenario where a developer in the tooling account needs emergency access to a production spoke account? Design a JIT access mechanism that creates temporary assume-role access, requires approval, and is automatically revoked after 4 hours.

Q14
5

Architecture — Microservices Communication Patterns

Hard

You're designing a system where Service A calls Service B synchronously. Service B is sometimes slow (3-10 second response). This causes Service A's thread pool to exhaust, causing cascading failures. Redesign the communication pattern.

 **Testing:** Circuit breaker pattern on AWS, async communication, timeouts, bulkhead pattern, SQS/Step Functions

Key Answer Points:

- Root cause: synchronous coupling. Service A blocks waiting for B. Under load, all threads block on slow B calls. Thread pool exhausted. A crashes even though A is healthy
- Option 1 — Timeout + Circuit Breaker: Service A sets strict timeout (1s). After N failures, circuit breaker opens — fails fast for X seconds without calling B. Library: resilience4j, Hystrix. AWS: no native circuit breaker — implement in code or use App Mesh
- Option 2 — Async via SQS: A puts request in SQS queue, immediately returns 'processing'. B consumes from queue. A polls or gets webhook callback when done. No thread exhaustion. B's slowness = longer processing time, not A failure
- Option 3 — Step Functions: orchestrate the A → B call as a workflow. Step Functions has built-in retry, timeout, and error handling. A triggers a Step Functions execution and returns immediately
- Option 4 — Service Mesh (App Mesh/Istio): circuit breaking and retry logic in the Envoy sidecar — no code changes needed
- Bulkhead pattern: separate thread pools for different downstream calls. B's slowness only exhausts B's thread pool, not A's main pool
- Idempotency key: for async patterns, include idempotency key in message. If B processes same request twice (retries), it's a no-op
- SLA discussion: if B must respond in < 1s for A to function, that's a contract. Document it. Alert when B p99 approaches 1s
- Design recommendation: SQS async for non-real-time operations. Sync with timeout + circuit breaker for real-time with graceful degradation
- Monitoring: track B's p99 latency from A's perspective (end-to-end), not just B's self-reported latency

 **Follow-up:** How does the ambassador pattern differ from a service mesh for implementing circuit breaking? In what environments is an ambassador proxy more practical than deploying a full service mesh?

Q14
6**WAF & Shield — DDoS Response****Hard**

During a major product launch, your website comes under a Layer 7 HTTP flood attack: 500,000 requests/second from 10,000 different IPs, all targeting valid URLs. CloudFront is in front. How do you respond in real time?

 **Testing:** AWS WAF rate limiting, Shield Advanced, CloudFront distributions, emergency response

 Key Answer Points:

- Immediate detection: CloudWatch dashboard shows request spike. WAF sampled requests show 500K rps from distributed IPs with matching User-Agent or header pattern
- Immediate action 1: WAF rate-based rule on CloudFront — block IPs exceeding 2,000 req/10min. Creates automatic block list with 5-minute expiry
- Immediate action 2: if attack has common User-Agent or header: WAF string match rule to block the pattern. Override to BLOCK
- CloudFront geo restriction: if attack from specific countries not in your user base — apply geo restriction immediately
- Shield Advanced: if subscribed, contact AWS DDoS Response Team (DRT) immediately. DRT can write custom WAF rules and has access to CloudFront infrastructure controls not available to customers
- Shield Advanced SRT: available 24/7. Escalate by creating a Shield Advanced support case
- WAF labels: use WAF managed rule groups (Bot Control, Account Takeover Prevention) that already score suspicious requests
- Rate limiting strategy: protect critical paths most aggressively (/api/auth, /checkout). Static assets less critical
- Post-incident: analyze WAF logs in S3 + Athena. Find attack signature. Create permanent rule for similar pattern
- Architecture lesson: CAPTCHA for suspected bots (WAF CAPTCHA action), not just block — allows legitimate users through

 **Follow-up:** Shield Advanced charges a \$3,000/month base fee plus data transfer costs during attacks. How do you justify this cost to management versus using WAF rate-based rules alone?

Q14
7**EKS — Cluster Upgrade Strategy****Hard**

You need to upgrade your production EKS cluster from Kubernetes 1.27 to 1.29 (skip 1.28 or upgrade sequentially). You have 50 nodes, 200 deployments, and several third-party operators (cert-manager, ExternalDNS, NGINX Ingress). What's your strategy?

 **Testing:** EKS version upgrade path, add-on compatibility, node drain mechanics, rollback planning

 Key Answer Points:

- EKS upgrade path: cannot skip minor versions. Must go 1.27 → 1.28 → 1.29. Each upgrade is a separate operation
- Step 1: check API deprecations. Use kubectl convert and Pluto tool to find deprecated API versions in your manifests. Kubernetes 1.28/1.29 removed several beta APIs
- Step 2: update third-party operators FIRST (or in tandem). cert-manager, ExternalDNS, NGINX Ingress have minimum K8s version requirements. Check compatibility matrix for each
- Step 3: upgrade control plane only (EKS managed). Control plane upgrades in-place. Takes 15-25 minutes. During this window: API server is briefly unavailable, existing pods continue running
- Step 4: upgrade managed add-ons (kube-proxy, CoreDNS, VPC CNI) via EKS managed add-ons or manually. These must match control plane version
- Step 5: upgrade node groups. For Managed Node Groups: rolling update — new nodes launched, old nodes cordoned and drained, pods rescheduled. For Karpenter: node pool drift handling
- PodDisruptionBudgets: all critical Deployments should have PDB with minAvailable: 1 or maxUnavailable: 0. Node drain respects PDB — won't drain if it would violate PDB
- Testing: upgrade staging cluster first. Run full test suite. Identify issues for 2 weeks before prod upgrade
- Rollback: EKS control plane can't be downgraded. Rollback = fix forward. This is why staging first is critical

- Blue/Green cluster upgrade: create new 1.29 cluster, migrate workloads using GitOps, cutover via DNS. More work but easier rollback

 **Follow-up:** What happens to pods currently running on a node during an EKS managed node group upgrade? Are they drained gracefully, and how does Kubernetes guarantee workloads continue serving traffic during the drain?

Q14
8

Serverless — Lambda Architecture at Scale

Hard

You're designing a serverless API that needs to handle: 10,000 rps average with 100,000 rps peak (10x flash sale), < 100ms p99 latency, global users, and must handle user state (shopping cart, preferences). Design the full architecture.

 **Testing:** Lambda scaling limits, DynamoDB design, Global Accelerator, latency optimization, stateless Lambda

✓ Key Answer Points:

- Traffic routing: AWS Global Accelerator → nearest AWS edge → API Gateway (HTTP API, lower latency than REST) → Lambda
- Lambda scaling: HTTP API can handle 100,000 rps. Lambda scales to thousands of concurrent executions. Account burst limit: 3,000 in large regions. Request quota increase BEFORE the flash sale
- Cold start mitigation: Provisioned Concurrency pre-warms 500 Lambda instances before flash sale. Keeps at minimum 200 warm post-sale
- State: Lambda is stateless — cart and preferences in DynamoDB (not Lambda memory). DynamoDB On-Demand for unpredictable flash sale traffic
- Session: JWT token in each request (stateless auth). Lambda validates JWT locally (no external call). Secrets Manager for JWT signing key (cached in Lambda initialization)
- DAX: DynamoDB Accelerator in front of DynamoDB. Cart reads from DAX (microseconds). Writes go to DynamoDB directly (strong consistency for inventory)
- CloudFront: cache product catalog responses (TTL: 60s). Reduces Lambda invocations for catalog queries by 90%
- Lambda Layers: shared dependencies (SDK, utils) in Lambda Layer — reduces package size, faster cold starts, shared across Lambdas
- Async processing: order confirmation email → SQS → Lambda (async). Inventory update → DynamoDB conditional write → EventBridge → downstream
- Observability: Lambda Powertools (structured logging, X-Ray tracing, metrics). CloudWatch dashboard for: rps, p99 latency, throttle count, error rate, cold start count

 **Follow-up:** Lambda has a 15-minute maximum execution timeout. If a flash sale creates a surge where some Lambda invocations queue and wait more than 15 minutes to even start processing, what happens? How do you design for this edge case?

Q14
9

Multi-Cloud — AWS + On-Premises Connectivity

Medium

Your organization is migrating to AWS from an on-premises data center. During the migration (18 months), both environments must coexist. Applications in AWS need to reach on-prem databases. On-prem applications need to call new AWS microservices. Design the hybrid connectivity.

 **Testing:** Direct Connect, Site-to-Site VPN, hybrid DNS, latency considerations, failover

✓ Key Answer Points:

- Direct Connect: dedicated 1-10 Gbps private connection between on-prem and AWS. Low latency (<10ms), consistent bandwidth. Lead time: 1-3 months to provision. Needed for 18-month migration
- Site-to-Site VPN: IPsec tunnel over internet. Available in hours. Higher latency (~50ms), bandwidth limited (~1.25 Gbps). Use as backup to Direct Connect
- Active/Passive: Direct Connect primary, VPN secondary. BGP route preference: Direct Connect AS path

shorter → preferred

- Transit Gateway + Direct Connect Gateway: Direct Connect Gateway connects to TGW. All VPCs connect to TGW. Single Direct Connect serves all VPCs
- Hybrid DNS: on-prem DNS resolves AWS private hostnames? Route 53 Resolver Endpoints: Inbound Endpoint (on-prem forwards AWS DNS queries to this IP) + Outbound Endpoint (AWS queries on-prem DNS for on-prem hostnames)
- IP addressing: ensure no CIDR overlap between on-prem and VPC. Remap if needed with AWS NAT
- Security: Security Groups on AWS side allow on-prem CIDR ranges. On-prem firewall allows AWS VPC CIDR
- Latency budget: Direct Connect adds ~10ms vs pure AWS-to-AWS. Design application timeouts accounting for this
- Data transfer costs: Direct Connect data transfer out: \$0.02/GB (vs internet: \$0.09/GB). For large data migrations, Direct Connect pays for itself quickly
- Migration cutover: run both environments, use DNS to shift traffic, on-prem becomes secondary, then decommission

 **Follow-up:** How do you handle the DNS resolution for an on-premises application calling an AWS API Gateway URL that resolves to a private IP via Route 53 Private Hosted Zone? Walk through the DNS resolution chain.

Q150

Interview — Architecture Whiteboard

Hard

Final question: Design a production-ready, scalable, highly-available food delivery platform on AWS. Key requirements: real-time order tracking (WebSocket), restaurant menu management, payment processing, driver location updates (GPS every 5 seconds from 10,000 active drivers), 99.99% uptime, GDPR compliance (EU data residency). High-level design, service choices, and why.

 **Testing:** End-to-end architecture design, service selection justification, trade-offs, compliance

✓ Key Answer Points:

- Compute: EKS (Fargate for serverless, Karpenter for scaling) for microservices (menu, order, payment). Lambda for event-driven (notifications, webhooks)
- Real-time WebSocket: API Gateway WebSocket API (fully managed, scales to 1M concurrent connections). Orders → WebSocket → customer app. Driver location updates via WebSocket to order tracking service
- Driver GPS (50,000 updates/sec = 10K drivers × 5/sec): Kinesis Data Streams (not SQS — need ordered per-driver, replay capability). Consumer: ECS service updates DynamoDB driver location table. DynamoDB TTL: purge old location records
- Database: DynamoDB (orders — high write throughput, flexible schema), Aurora PostgreSQL (restaurants, menus — relational, complex queries), ElastiCache Redis (driver locations for real-time map queries, rate limiting)
- Payment: AWS PrivateLink to Stripe/payment processor. Secrets Manager for API keys. Step Functions for payment workflow with retries + compensation
- Messaging: SQS for order notifications, EventBridge for system events (OrderPlaced → kitchen, driver dispatch, inventory)
- GDPR / EU data residency: VPC + services exclusively in eu-west-1 and eu-central-1. DynamoDB Global Tables between EU regions only. SCPs deny creating resources outside EU regions. Data classification with Macie
- CDN: CloudFront for restaurant images, static assets. Route 53 geolocation routing
- Observability: kube-prometheus-stack on EKS, CloudWatch for managed services, X-Ray for distributed tracing, OpenSearch for logs
- 99.99% uptime: Multi-AZ for everything, multi-region active-passive for Regional failure, PodDisruptionBudgets, circuit breakers between services, chaos engineering quarterly

 **Follow-up:** You designed DynamoDB for orders. A data analyst needs to run complex SQL queries joining orders with restaurant data and driver performance data for business intelligence. DynamoDB doesn't support JOINS. How do you build the analytics layer without impacting the OLTP performance?



Series Complete — 150 Questions Across 3 Documents

All questions are scenario-based. Every answer has follow-up probes. Read across all 3 parts for full coverage.

Part	Questions	Core Services Covered
Part 1 (Q1–Q50)	50 questions	EC2, VPC, ALB/NLB, ASG, RDS basics, S3 basics, Lambda, Route 53, CloudFront, Security Groups, EKS basics
Part 2 (Q51–Q100)	50 questions	EKS advanced, IAM deep-dive, TGW, PrivateLink, Network Firewall, S3 advanced, RDS advanced, Multi-AZ, ML on AWS
Part 3 (Q101–Q150)	50 questions	SQS/SNS/EventBridge, Step Functions, ECS/Fargate, DynamoDB, ElastiCache, CloudFormation/CDK, Cost, CloudWatch/X-Ray, Secrets, CI/CD, Advanced Architecture